

UNIVERSITY CAREER FAIR

Company: Twitter

Round: Online Assessment

Year: 2021

Difficulty: Medium

Topic: Greedy Algorithm, Sorting, Interval Scheduling

Source: https://algo.monster/problems/twitter_oa_university_career_fair

Problem Description:

Imagine that several companies want to give presentations at a university Career Day.

Each company has:

- A starting time — when its presentation can begin.
- A duration — how long the presentation will last.

The university can have only one presentation at a time.

Our goal is to choose the presentations in such a way that we can conduct the maximum possible number of presentations.

Important Point

A presentation that starts at time start and lasts for duration minutes will finish at:

finish time = start + duration

Two presentations can be scheduled one after another if the second presentation starts when or after the first one finishes.

For example:

Company A: 1 → 3

Company B: 3 → 5

These do not overlap, because Company B starts exactly when Company A finishes.

Input Format:

We are given two array

Contains the starting time of every company

Contains the duration of every company

Output Format:

Return an integer representing the maximum number of presentations that can be scheduled without overlapping.

Sample Input:

starts = [1, 3, 3, 5, 7]

durations = [2, 2, 1, 2, 1]

Sample Output:

Maximum Number of Presentation = 4

Explanation:

Let's find the time interval of each company.

Company Start Duration Finish

0	1	2	3
1	3	2	5
2	3	1	4
3	5	2	7
4	7	1	8

So the presentations are:

Company 0 → 1 to 3

Company 1 → 3 to 5

Company 2 → 3 to 4

Company 3 → 5 to 7

Company 4 → 7 to 8

We can choose:

Company 0 → 1 to 3

Company 2 → 3 to 4

Company 3 → 5 to 7

Company 4 → 7 to 8

There is no overlap between these presentations.

Therefore, we can conduct:

4 presentations

We could also choose Company 1 instead of Company 2:

Company 0 $\rightarrow$ 1 to 3
Company 1 $\rightarrow$ 3 to 5
Company 3 $\rightarrow$ 5 to 7
Company 4 $\rightarrow$ 7 to 8

That also gives 4.

Approach to Solve This Problem:

This is a classic interval scheduling problem.

The main idea is:

Always choose the presentation that finishes earliest.

Why?

Suppose we have two presentations that we could choose next:

A $\rightarrow$ 1 to 5
B $\rightarrow$ 2 to 3

If we choose A, the next presentation can only start at 5.

But if we choose B, the next presentation can start at 3.

So choosing the presentation that finishes earlier leaves more time for other presentations.

This is called a Greedy Algorithm.

Step 1: Create the presentation intervals

For every company, calculate:

$\text{end} = \text{starts}[i] + \text{durations}[i]$

For our example:

Company 0 $\rightarrow$ [1, 3]
Company 1 $\rightarrow$ [3, 5]
Company 2 $\rightarrow$ [3, 4]
Company 3 $\rightarrow$ [5, 7]
Company 4 $\rightarrow$ [7, 8]

Step 2: Sort by finish time

We sort the presentations according to their ending time.

For this example, they become:

[1, 3]
[3, 4]
[3, 5]
[5, 7]
[7, 8]

Step 3: Pick presentations greedily

We keep track of the time when the last selected presentation finished.

Initially:

lastFinish = 0

Now check each presentation.

[1, 3]

It starts at 1, which is after lastFinish = 0.

So we select it.

count = 1
lastFinish = 3
[3, 4]

It starts at 3.

Since:

$3 \geq 3$

there is no overlap.

Select it.

count = 2
lastFinish = 4
[3, 5]

It starts at 3, but our previous presentation ends at 4.

$3 < 4$

So it overlaps.

We skip it.

[5, 7]

It starts at 5.

$5 \geq 4$

So we select it.

count = 3
lastFinish = 7
[7, 8]

It starts at 7.

$7 \geq 7$

So we select it.

count = 4

Therefore, the answer is:

4

Why the Greedy Approach Works

The important idea is that we want to leave as much time as possible for future presentations.

If one presentation finishes at 4 and another finishes at 7, choosing the one that finishes at 4 is better because we can start another presentation much earlier.

So at every step:

Choose the compatible presentation with the earliest finishing time.

This gives us the maximum number of non-overlapping presentations.

Java Code:

```
1  import java.util.*;
2
3  public class Main {
4
5      static class Presentation {
6          int start;
7          int end;
8
9          Presentation(int start, int end) {
10             this.start = start;
11             this.end = end;
12         }
13     }
14
15     public static int maxPresentations(int[] starts, int[] durations) {
16
17         int n = starts.length;
18
19         Presentation[] presentations = new Presentation[n];
20
21         // Create intervals
22         for (int i = 0; i < n; i++) {
23             int end = starts[i] + durations[i];
24
25             presentations[i] =
26                 new Presentation(starts[i], end);
27         }
28
29         // Sort by finishing time
30         Arrays.sort(presentations, (a, b) -> a.end - b.end);
31
32         int count = 0;
33         int lastFinish = 0;
34     }
```

```
3  public class Main {
15     public static int maxPresentations(int[] starts, int[] durations) {
16
17         // Select non-overlapping presentations
18         for (Presentation p : presentations) {
19
20             if (p.start >= lastFinish) {
21                 count++;
22                 lastFinish = p.end;
23             }
24         }
25
26         return count;
27     }
28
29     // Run/Debug
30     public static void main(String[] args) {
31
32         Scanner sc = new Scanner(System.in);
33
34         int n = sc.nextInt();
35
36         int[] starts = new int[n];
37         int[] durations = new int[n];
38
39         for (int i = 0; i < n; i++) {
40             starts[i] = sc.nextInt();
41         }
42
43         for (int i = 0; i < n; i++) {
44             durations[i] = sc.nextInt();
45         }
46
47         System.out.println(maxPresentations(starts, durations));
48
49         sc.close();
50     }
51 }
```

Solution:

The problem is basically asking us to select the maximum number of non-overlapping intervals.

We first convert every company into an interval:

$[start, start + duration]$

Then we sort all intervals by their finish time.

Starting from the earliest finishing presentation, we keep selecting a presentation whenever:

$start \geq lastFinish$

This means it does not overlap with the presentation we selected previously.

The reason this works is simple:

A presentation that finishes earlier leaves more free time for the presentations that come later.

For the given example, we can schedule 4 presentations.

Therefore:

Output = 4

Understanding the Important Part of the Code:

1. Creating an Interval

```
int end = starts[i] + durations[i];
```

This calculates when a company's presentation finishes.

For example:

$start = 3$

$duration = 2$

$end = 3 + 2 = 5$

So the presentation is:

$3 \rightarrow 5$

2. Storing the Presentation

```
Presentation[] presentations = new Presentation[n];
```

Each presentation stores:

start
end

This makes it easier to sort and compare presentations.

3. Sorting by Finish Time

`Arrays.sort(presentations, (a, b) -> a.end - b.end);`

This is the most important part of the greedy approach.

We arrange presentations from the one that finishes earliest to the one that finishes latest.

4. Checking for Overlap

`if (p.start >= lastFinish)`

This checks whether the current presentation can start after the previous selected presentation has finished.

For example:

Previous: $1 \rightarrow 4$

Current: $4 \rightarrow 6$

Since:

$4 \geq 4$

they don't overlap.

So we can select the current presentation.

5. Updating the Answer

`count++;`
`lastFinish = p.end;`

When we select a presentation:

- Increase the number of presentations.
- Remember its finishing time.

Complexity Analysis:

Time Complexity: $O(n \log n)$

Creating all intervals takes $O(n)$.

Sorting the presentations takes:

$O(n \log n)$

Going through the sorted presentations takes $O(n)$.

Therefore, the overall complexity is:

Time Complexity = $O(n \log n)$

Space Complexity: $O(n)$

We create an array containing n presentation objects.

Therefore:

Space Complexity = $O(n)$